Security Design Recommendations

Mobile Banking Application

Application Security Architecture Report

By: Rahaf Albarakati

Executive Summary

This report presents security design recommendations for a mobile banking application. Banking applications handle highly sensitive information, including customer identities, account balances, transaction history, payment information, and financial transfers. Because they are attractive targets for cyberattacks, security must be incorporated into the application architecture from the beginning rather than added later.

The following recommendations focus on protecting customer data, preventing unauthorized access, securing communication between the mobile application and backend services, and improving the application's overall resilience against common cyber threats.

Architecture Overview

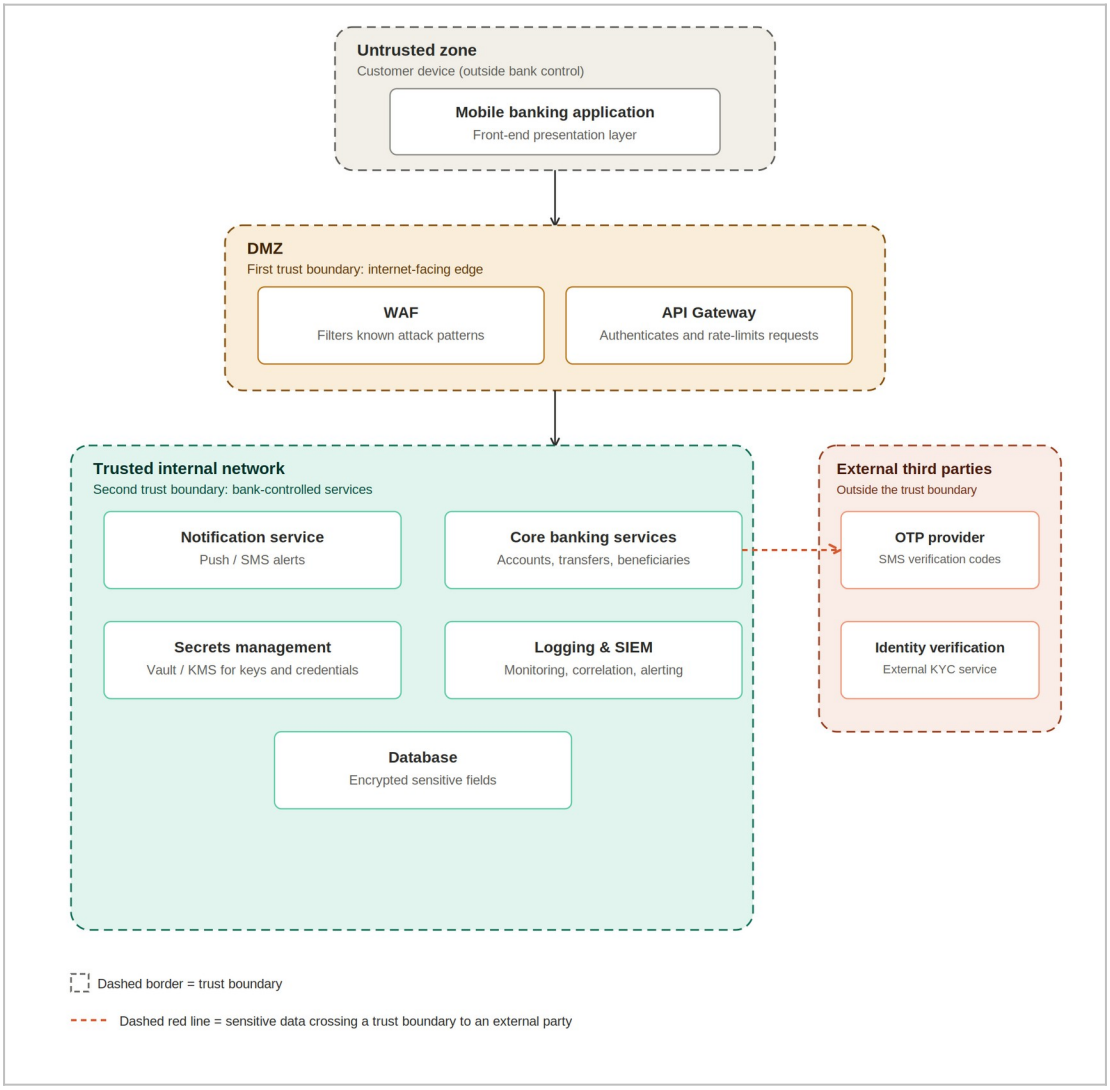


Figure 1. Request path and trust boundaries between the mobile application, the API layer, the trusted internal network, and external third parties.

1. Authentication

Authentication is the process of verifying that a user is who they claim to be.

Why did I choose this solution?

To ensure that only the legitimate account owner can log in and access banking services.

What could happen if it is ignored?

- Unauthorized users could access customer accounts.
- Stolen passwords could be used to perform fraudulent transactions.

How should the development team implement it?

- Create a login page where users enter their email address and password.
- Send the login credentials securely to the authentication server using HTTPS.
- The authentication server verifies the user's credentials against the stored account information.
- If authentication is successful, the server generates an **Access Token** and returns it to the Android application.
- Store the Access Token securely on the device, as described in **Section 9: Secure Storage**.
- Use the Access Token in the **Authorization** header for all subsequent API requests instead of sending the user's password.
- Set the Access Token to expire after 15 minutes. When it expires, require the application to obtain a new Access Token using a Refresh Token.
- Enable **Multi-Factor Authentication (MFA)** to provide an additional layer of security during login.
- Implement **Rate Limiting** and temporarily lock the account after multiple failed login attempts to mitigate brute-force attacks.
- Delete the stored Access Token immediately when the user logs out to prevent unauthorized reuse.
- Support **Biometric Authentication** (fingerprint or face unlock) through Android's **BiometricPrompt** API after the first successful login.

2. Authorization

Ensure that the user can access only the data and functions they are authorized to use after authentication.

Why did I choose this solution?

To ensure users can only access their own accounts and authorized banking services.

What could happen if it is ignored?

- Users could access another customer's account.
- Unauthorized transactions or data exposure could occur.

How should the development team implement it?

- Every API request must include a valid **Access Token**.
- The backend validates the token before processing the request.
- After validating the token, the backend identifies the user and their permissions.
- Users can only access their own bank accounts and personal information.
- The backend verifies authorization for every sensitive operation, such as money transfers, adding beneficiaries, or changing account information.
- If a user attempts to access another customer's data or perform an unauthorized action, the request is denied. The server returns:
  - **401 Unauthorized** if the token is invalid or expired.

    - **403 Forbidden** if the user is authenticated but does not have permission to perform the requested action.
  - All authorization failures and suspicious activities are logged for security monitoring and auditing.

3. Session Management

Session management controls how a user's authenticated state is created, maintained, and ended after login, so that access stays tied to the legitimate user for only as long as it should.

Why did I choose this solution?

To ensure that active sessions and their tokens cannot be hijacked or reused after they should no longer be valid.

What could happen if it is ignored?

- Attackers could reuse stolen session tokens.

How should the development team implement it?

- ***Store Tokens Securely***

  Both tokens must be stored encrypted on the device, never in plain text, as described in Section 9: Secure Storage.

  The **Refresh Token** is more sensitive because it can generate new Access Tokens. It should also be encrypted and protected by the Android Keystore. Whenever possible, it should never be accessible to other applications or stored in plain text.

- ***Use Short-Lived Access Tokens***

  Access Tokens should have a short lifetime (15 minutes, as defined in Section 1). If an attacker steals an Access Token, it will only be usable for a limited time.

- **Rotate Refresh Tokens**

  Whenever a Refresh Token is used to obtain a new Access Token, the server should invalidate the old Refresh Token and issue a new one.

  This prevents attackers from reusing previously stolen Refresh Tokens.

- **Invalidate Sessions**

  The server should immediately invalidate all active sessions when:

- The user logs out.
- The password is changed.
- Suspicious activity is detected.
- The account is locked.
- The user chooses "Log out from all devices."

Once invalidated, the associated Access Tokens and Refresh Tokens should no longer be accepted.

o Automatic Session Timeout

The application should automatically log users out after a period of inactivity and require authentication again before allowing further access.

o Step-Up Authentication for Sensitive Operations

Logging in once should not be sufficient for high-risk banking operations.

The application should require additional authentication before actions such as:

- Adding a new beneficiary.
- Transferring money.
- Increasing transfer limits.
- Changing security settings.
- Updating the registered phone number.
- Resetting the transaction PIN.

Additional verification may include:

- Biometric authentication.
- One-Time Password (OTP).
- Device PIN.
- Push approval.

o Detect Suspicious Sessions

The server should continuously monitor active sessions for suspicious behavior, such as:

- Login from a new device.
- Login from a different geographic location.
- Impossible travel (two distant locations within a short period).
- Multiple failed authentication attempts.
- Unusual transaction patterns.

If suspicious activity is detected, the session should be terminated immediately, and the customer should be notified.

o Protect Against Device Theft

If a customer reports a lost or stolen device, the bank should be able to remotely revoke all active sessions associated with that device.

- **Bind Sessions to the Device**

  Sessions should be associated with trusted device information, such as the registered device identifier.

  If the same token is used from a different device, the server should reject the request or require additional verification.

4. API Security

Why did I choose this solution?

To secure communication between the mobile application and backend services while protecting APIs from unauthorized access and abuse.

What could happen if they are ignored?

- Attackers may access other customers' accounts or transaction history.
- Unauthorized money transfers could be performed.
- Sensitive banking information could be exposed.
- Automated attacks could overload the API.
- Duplicate financial transactions may occur.
- Attackers could modify protected fields or bypass application logic.

How should the development team implement it?

1. Enforce Object-Level Authorization (Prevent IDOR/BOLA)

The server must verify that the authenticated user is authorized to access every requested resource. Authorization should be checked for each account, beneficiary, transaction, or payment object instead of trusting identifiers received from the client.

2. Prevent Mass Assignment

The API should only allow specific fields that are explicitly expected for each request. Sensitive fields such as account balance, account role, account status, user privileges, or internal identifiers must never be accepted directly from client input.

3. Validate Request Size

Set maximum limits for request body size, uploaded files, and the number of objects submitted in a single request. This prevents resource exhaustion and denial-of-service attacks caused by oversized requests.

4. Validate Input Data

Validate every parameter received by the API.

The server should verify:

- Required fields
- Data type
- Length
- Format
- Allowed values
- Numeric ranges
- Business rules

Invalid requests should be rejected before any processing occurs.

5. Validate Content-Type

Only accept supported content types such as **application/json**. Reject unexpected content types or malformed payloads to reduce the attack surface.

6. Protect Financial Transactions with Idempotency Keys

Operations such as money transfers, bill payments, or card payments should require an **Idempotency Key**.

If the same request is received multiple times due to retries, network issues, or malicious replay attempts, the server should process it only once and return the original response instead of executing the transaction again.

7. Apply Rate Limiting

Limit the number of requests per user, IP address, device, or API key within a specific time period.

Critical endpoints such as login, OTP verification, password reset, and money transfer should have stricter rate limits to reduce brute-force and abuse attacks.

8. Deploy an API Gateway

Use an API Gateway as the single entry point for backend services.

The API Gateway should:

- Authenticate incoming requests.
- Validate JWT access tokens.
- Apply rate limiting.
- Filter malicious requests.

- Log API activity.
- Integrate with a Web Application Firewall (WAF).

9. Perform Server-Side Validation

Never rely on validation performed by the mobile application.

All authorization checks, input validation, transaction limits, account ownership verification, and business rules must be enforced again on the server because attackers can interact directly with the API without using the official mobile application.

10. Secure API Communication

Require HTTPS with TLS for all API communication, disable insecure protocols and weak cipher suites, and ensure certificates are properly validated to protect data in transit.

5. Data Protection (Encryption in Transit and at Rest)

Why did I choose this solution?

To ensure that sensitive banking information remains confidential, maintains its integrity, and is protected from unauthorized access during transmission and while stored. Effective data protection also requires secure encryption key management, as the security of encrypted data depends on both the encryption algorithm and the protection of its cryptographic keys.

What could happen if it is ignored?

- Attackers could intercept sensitive data through Man-in-the-Middle (MITM) attacks.
- Customer information could be exposed if databases or backups are compromised.
- Stolen encryption keys could allow attackers to decrypt sensitive information.
- Unauthorized users or services could gain access to confidential banking data.

How should the development team implement it?

Protect Data in Transit

- Use **HTTPS** with **TLS 1.2 or TLS 1.3** for all communication between the mobile application and backend services.
- Implement **Certificate Pinning** in the mobile application to reduce the risk of Man-in-the-Middle (MITM) attacks.
- Disable insecure protocols and weak cipher suites.

Protect Data at Rest

- Encrypt sensitive data stored in databases, file systems, and backups using **AES-256**.
- Apply **field-level encryption** to highly sensitive information such as account numbers, card numbers, national IDs, and personally identifiable information (PII), rather than relying only on full-database encryption.
- Encrypt database backups before storage or transfer.

Secure Key Management

- Store encryption keys in a dedicated **Key Management System (KMS)** or **Hardware Security Module (HSM)**. Encryption keys must never be stored in application source code, configuration files, or databases.
- Rotate encryption keys periodically and immediately replace them if compromise is suspected.
- Restrict the use of encryption keys based on the **principle of least privilege**, ensuring that only authorized services and personnel can decrypt sensitive information.
- Maintain audit logs for all key management operations, including key creation, usage, rotation, and deletion.
- Never store encryption keys together with the encrypted data.

6. Secrets Management

Why did I choose this solution?

To prevent sensitive credentials from being exposed in the application.

What could happen if it is ignored?

- API keys or encryption keys could be extracted from the application.
- Attackers could access backend services.

How should the development team implement it?

- Never hardcode secrets in the source code.
- Store secrets in a secure backend secret manager.
- Keep encryption keys inside secure hardware or cloud key management services.
- Never ship backend credentials such as database passwords or private API keys inside the mobile app; anything included in the APK can be extracted through reverse engineering.
- Have the app obtain short-lived, per-user tokens from the backend instead of relying on static API keys.
- Scan source code and CI/CD pipelines automatically for leaked secrets before every release.
- Rotate secrets periodically and revoke any secret immediately if exposure is suspected.
- Restrict access to secrets by the principle of least privilege and audit every access.

7. Logging and Monitoring

Why did I choose this solution?

To detect attacks, investigate incidents, and monitor suspicious activities.

What could happen if it is ignored?

Security incidents may go unnoticed.

How should the development team implement it?

- Log **failed login attempts**, including repeated authentication failures that may indicate brute-force attacks.
- Log **successful logins**, including the device identifier, IP address, timestamp, and approximate location when permitted.
- Log **new device registrations** and notify the customer when a new device is used to access the account.
- Log **changes to sensitive account information**, such as password changes, phone number updates, email changes, and security settings.
- Log **high-risk financial operations**, including adding a new beneficiary, money transfers, increasing transfer limits, and large-value transactions.
- Log **failed Access Token or Refresh Token validation**, as these may indicate token tampering or replay attempts.
- Log **unauthorized access attempts**, including requests that fail authorization checks such as IDOR/BOLA attempts.
- Detect and log **unusual account activity**, such as logins from unfamiliar devices, abnormal geographic locations, impossible travel scenarios, or unusual transaction patterns.
- Monitor all security logs using a **Security Information and Event Management (SIEM)** solution to correlate events and detect attacks in real time.
- Generate automatic alerts for high-risk security events so that the security team can investigate immediately.
- Never log sensitive information such as passwords, PINs, CVVs, OTPs, Access Tokens, Refresh Tokens, or encryption keys.

8. Error Handling

Why did I choose this solution?

To prevent attackers from learning internal system details.

What could happen if it is ignored?

Detailed error messages could reveal database structures, server information, or application logic.

How should the development team implement it?

- Return generic, user-facing error messages that do not expose internal details.
- Use appropriate HTTP status codes (for example, 400 Bad Request, 401 Unauthorized, 403 Forbidden) while keeping technical details in server-side logs only.
- Use one generic message for failed logins (for example, “Invalid email or password”) so attackers cannot tell whether an account exists.
- Never return stack traces, SQL errors, server versions, or internal file paths in API responses.
- Log full technical details on the server with a correlation ID, and show the user only that ID for support purposes.
- Use a global exception handler so unexpected errors fail safely instead of exposing raw output.
- Disable debug mode and verbose error output in production builds.

9. Secure Storage

Why did I choose this solution?

To protect sensitive data stored on the user's device.

What could happen if it is ignored?

- Sensitive data may be exposed if an attacker gains access to the device, application storage, backups, or database systems.
- Session hijacking could occur.

How should the development team implement it?

- Use the **Android Keystore** to securely generate and store cryptographic keys. Do not store sensitive application data directly in the Keystore.
- Store sensitive application data in **Encrypted SharedPreferences** or **Encrypted DataStore**, encrypted using keys protected by the Android Keystore.
- **Never store** highly sensitive information such as:
    - Customer PINs.
    - CVV values.
    - Card magnetic stripe data.
    - One-Time Passwords (OTPs).
    - User passwords.

- Only store data that is necessary for application functionality. Sensitive information such as account numbers or card numbers should only be stored when there is a clear business need, and then only in encrypted or masked form.
- Mask sensitive information when displaying it to the user (for example, display only the last four digits of an account or card number).
- Delete sensitive data immediately when it is no longer required, such as after logout or account removal.
- Prevent sensitive information from being written to logs, cache files, or temporary storage.

10. Implementation Priority Matrix

The recommendations in this report do not carry equal urgency. The table below consolidates the key recommendations from every section into a single roadmap, ranked by priority, and assigns an owner and a concrete validation method to each one so that implementation can be tracked and verified rather than treated as a general statement of intent.

Critical items must be implemented and verified before the application goes live. **High priority** items should be scheduled shortly after launch. **Recommended enhancement** items strengthen the overall security posture and can be planned into a later iteration.

Recommendation	Risk Addressed	Priority	Owner	Validation Method
Implement server-side object-level authorization on every account, beneficiary, and transaction endpoint (prevent IDOR/BOLA)	Unauthorized access to another customer's account or data	**Critical**	Backend Team	Attempt to access another user's account by modifying the account or resource ID and confirm the server rejects the request
Restrict input fields to an explicit allow-list to prevent mass assignment (block balance, role, status, and internal identifiers from client input)	Privilege escalation or unauthorized balance/status modification	**Critical**	Backend Team	Submit a request containing restricted fields (e.g., role, balance) and confirm they are ignored or the request is rejected
Require Multi-Factor Authentication (MFA) at login	Account takeover through credential theft	**Critical**	Backend + Mobile Team	Attempt login with valid credentials only and confirm access is blocked until the second factor is verified
Require step-up authentication (OTP, biometrics, or push approval) before high-risk actions such as transfers or adding beneficiaries	Fraudulent transactions through a hijacked or unattended session	**Critical**	Backend + Mobile Team	Attempt a high-risk action using only an active session token and confirm additional verification is required
Require an Idempotency Key for money transfers, bill payments, and card payments	Duplicate financial transactions from retries or replay attempts	**Critical**	Backend Team	Resend an identical transfer request twice with the same key and confirm it executes only once
Perform full server-side validation of every parameter (required fields, type, length, format, allowed values, ranges, business rules)	Injection attacks, negative amounts, or logically invalid requests	**Critical**	Backend Team	Submit out-of-range, malformed, or missing values and confirm the request is rejected before processing
Apply field-level encryption (AES-256)	Data disclosure if the	**Critical**	Backend +	Query the database

to sensitive data and manage keys through a dedicated KMS/HSM	database or backups are compromised		Security Team	directly and confirm sensitive fields cannot be read without the encryption key
Store all secrets (API keys, database credentials, encryption keys) in a secure secrets manager rather than in source code or config files	Credential exposure through source code, backups, or reverse engineering	**Critical**	Backend + DevOps	Scan the source code and compiled application for hardcoded secrets and confirm none are found
Apply stricter rate limiting on login, OTP verification, password reset, and money transfer endpoints	Brute-force and credential-stuffing attacks	**High**	Backend + Infra Team	Exceed the allowed number of attempts and confirm the server responds with 429 Too Many Requests
Deploy an API Gateway integrated with a Web Application Firewall (WAF) as the single entry point to backend services	Direct exposure of backend services and automated attack traffic	**High**	Infrastructure Team	Confirm backend services are unreachable except through the gateway, and known attack patterns are blocked by the WAF
Rotate Refresh Tokens on every use and invalidate the previous token	Reuse of stolen refresh tokens	**High**	Backend Team	Attempt to use a previously issued refresh token after rotation and confirm it is rejected
Implement certificate pinning in the mobile application	Man-in-the-middle (MITM) interception of API traffic	**High**	Mobile Team	Attempt a connection through an untrusted or proxy certificate and confirm the application rejects it
Bind sessions to registered device information and monitor for suspicious session behavior (new device, unusual location, impossible travel)	Session hijacking or account access from an unrecognized device	**High**	Backend Team	Reuse a valid token from a different, unregistered device and confirm additional verification is required
Monitor high-risk security events (failed logins, token failures, high-value transfers) through a centralized SIEM with automated alerting	Delayed detection of active attacks	**High**	Security Team / SOC	Generate a simulated high-risk event and confirm an alert is triggered and visible to the security team
Validate Content-Type headers and enforce request size and payload object-count limits	Resource exhaustion and denial-of-service through oversized or malformed requests	**Recommended**	Backend Team	Submit an oversized payload or unsupported content type and confirm the server rejects it with the correct status code
Return generic, non-technical error messages and mask sensitive data in the UI and application logs	Information disclosure that assists attackers or exposes data in logs	**Recommended**	Backend + Mobile Team	Trigger an application error and review logs to confirm no internal details or unmasked sensitive data are exposed

Conclusion

- A secure banking application requires multiple layers of security working together to protect customer information and financial assets. Strong authentication, robust authorization, secure session management, encrypted communications, secure storage,

protected APIs, effective secret management, and continuous monitoring collectively reduce cyber risks and strengthen the application's overall security posture.

- By implementing these recommendations during the design phase, the development team can build a banking application that complies with modern security best practices, protects sensitive financial data, and maintains customer trust while reducing the likelihood and impact of cyberattacks.

References

- **OWASP MASVS (Mobile Application Security Verification Standard)** — storage, cryptography, authentication, network, and platform controls (Sections 1, 3, 5, 9).
- **OWASP API Security Top 10 (2023)** — API1 Broken Object Level Authorization, API3 Broken Object Property Level Authorization, API4 Unrestricted Resource Consumption (Section 4).
- **OWASP Mobile Top 10 (2024)** — M1 Improper Credential Usage, M3 Insecure Authentication/Authorization, M9 Insecure Data Storage (Sections 1, 2, 6, 9).
- **NIST SP 800-63B Digital Identity Guidelines** — authentication and session management (Sections 1, 3).
- **OWASP Logging and Error Handling Cheat Sheets** — security logging and safe error responses (Sections 7, 8).